

Epic: API Logic Integration

Project: LegalEase

1. Epic Overview

Build and integrate the API layer that connects the LegalEase front end, business logic, retrieval pipeline and the selected AI model, so that every request flows through a secure, reliable and well-documented interface.

2. Background & Problem Statement

The model and architecture decisions need a working API layer to become a usable product. Without consistent request handling, validation, error handling and logging, the system cannot deliver dependable and auditable legal answers.

3. Goals & Objectives

- Define a clear API contract for all LegalEase services
- Integrate model calls, retrieval and business rules behind the API
- Ensure secure authentication and authorization on every endpoint
- Provide consistent error handling, logging and monitoring

4. Stakeholders

Role	Interest
Product Owner	Feature priorities, API behaviour
Backend Engineers	API design and implementation
Frontend Engineers	Stable contract to build against
ML Engineers	Model and retrieval integration
Security / QA	Access control, testing, audit

5. Scope

5.1 In Scope

- REST API design and OpenAPI specification
- Business logic layer and orchestration of model and retrieval calls
- Authentication, authorization and rate limiting
- Error handling, logging and audit trail
- Integration with third-party services, if any

5.2 Out of Scope

- Model selection and training (covered in a separate epic)
- UI/UX implementation
- Production infrastructure scaling

6. Assumptions & Constraints

- The model and architecture decisions are finalized before integration starts
- APIs must not log or expose confidential client data

- Contract changes must be versioned to avoid breaking clients

7. Key API Areas

Area	Purpose
Auth	Login, tokens, role-based access
Documents	Upload, parse, store, delete
Query / Chat	Send a legal question, get answer with citations
Analysis	Clause extraction, summarization, risk flags
Feedback	Capture user corrections and ratings
Admin	Usage, audit logs, configuration

8. Integration Logic Flow

- 1 Request received, authenticated and validated
- 2 Business rules applied (permissions, document access)
- 3 Retrieval service fetches relevant sources
- 4 Model is called with prompt and retrieved context
- 5 Response checked for citations and confidence
- 6 Result returned; request and outcome logged

9. Non-Functional Requirements

- Performance: target p95 latency defined per endpoint
- Reliability: retries, timeouts and graceful fallback if the model fails
- Security: encryption in transit, input validation, rate limiting
- Observability: structured logs, metrics and alerts

10. User Stories

- 1 As a frontend developer, I want a documented API contract so that I can build screens without waiting on the backend.
- 2 As a lawyer, I want to submit a question and receive an answer with citations through a single request.
- 3 As a backend engineer, I want model and retrieval calls orchestrated in one service layer so that the logic is easy to maintain.
- 4 As a security lead, I want every endpoint authenticated and every request logged for audit.
- 5 As a product owner, I want clear error messages and fallback behaviour when the model is unavailable.

11. Acceptance Criteria

- OpenAPI specification published and reviewed
- All endpoints require authentication and role checks
- Integration tests pass for the main flows
- Errors return consistent codes and messages
- Audit logs capture requests without exposing sensitive content

12. Deliverables & Milestones

Phase	Deliverable
1. Design	API contract and data models
2. Build	Core endpoints and business logic layer
3. Integrate	Model and retrieval connected end to end
4. Test	Integration, security and load testing
5. Document	API docs and handoff to frontend

13. Risks & Mitigation

Risk	Mitigation
Model API downtime or latency	Timeouts, retries, fallback model
Breaking contract changes	Versioned APIs, contract tests
Sensitive data in logs	Redaction and log review
Third-party service failure	Circuit breakers, graceful degradation

14. Dependencies

- Approved model and architecture decisions
- Retrieval / vector database available
- Auth provider and environment access

15. Success Metrics

- API success rate
- p95 latency per endpoint
- Test coverage on main flows
- Error rate in production

16. Open Questions

- Which auth method will be used (SSO, JWT, API keys)?
- Which third-party services must be integrated first?
- Sync or streaming responses for model answers?